

Programación en el kernel: `printk()` y user memory access

Diseño de Sistemas Operativos

Grado en Ing. de Computadores, 3er Curso

Depto. de Arquitectura de Computadores
Universidad de Málaga

Contenidos

```
asmlinkage long sys_my_get_jiffies(unsigned long __user *res)
{
  unsigned long cur_jiffies = (unsigned long) get_jiffies_64();

  printk(KERN_INFO "Contador de ticks= %lu", cur_jiffies);
  printk(KERN_INFO "ticks por segundo= %d", HZ);

  if(res) return( put_user(cur_jiffies, res) );
  return -EFAULT;
}
```

- `printk()` : Mensajes del kernel
- Acceso al espacio de memoria de usuario

Depurar en el kernel: `printk()`

```
printk(KERN_ALERT "DEBUG: Passed %s %d\n",__FUNCTION__,__LINE__);
```

- Formatos `printk()` ¿ `%f` ?

Tipo	Formato
int	%d ó %x
unsigned int	%u ó %x
long	%ld ó %lx
unsigned long	%lu ó %lx
long long	%lld ó %llx
unsigned long long	%llu ó %llx
size_t	%zu ó %zx
ssize_t	%zd ó %zx

https://www.kernel.org/doc/Documentation/printk-formats.txt

Depurar en el kernel: `printk()`

```
printk(KERN_ALERT "DEBUG: Passed %s %d\n",__FUNCTION__,__LINE__);
```

- Log levels

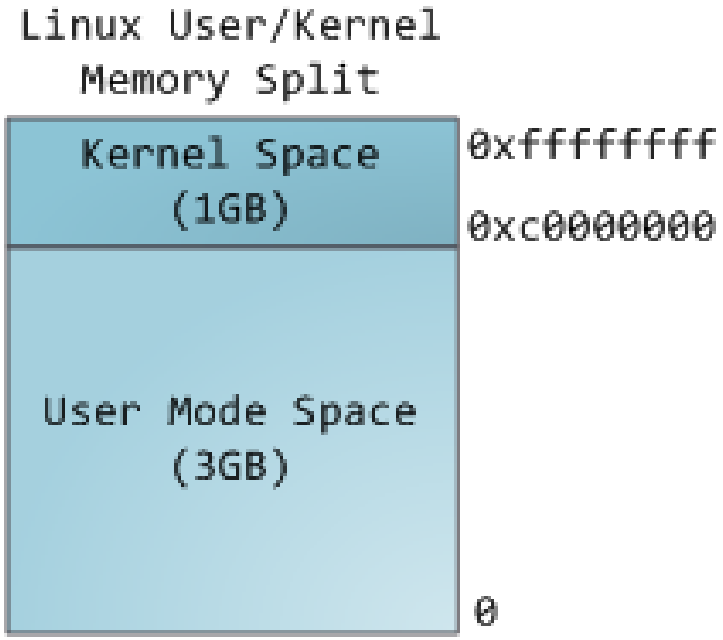
Name	String	Meaning
KERN_EMERG	"0"	Emergency messages, system is about to crash or is unstable
KERN_ALERT	"1"	Something bad happened and action must be taken immediately
KERN_CRIT	"2"	A critical condition occurred like a serious hardware/software failure
KERN_ERR	"3"	An error condition, often used by drivers to indicate difficulties with the hardware
KERN_WARNING	"4"	A warning, meaning nothing serious by itself but might indicate problems
KERN_NOTICE	"5"	Nothing serious, but notably nevertheless. Often used to report security events.
KERN_INFO	"6"	Informational message e.g. startup information at driver initialization
KERN_DEBUG	"7"	Debug messages
KERN_DEFAULT	"d"	The default kernel loglevel
KERN_CONT	""	"continued" line of log printout (only done after a line that had no enclosing \n)

- Visualizar mensajes del kernel: **`dmesg`**

https://elinux.org/Debugging_by_printing

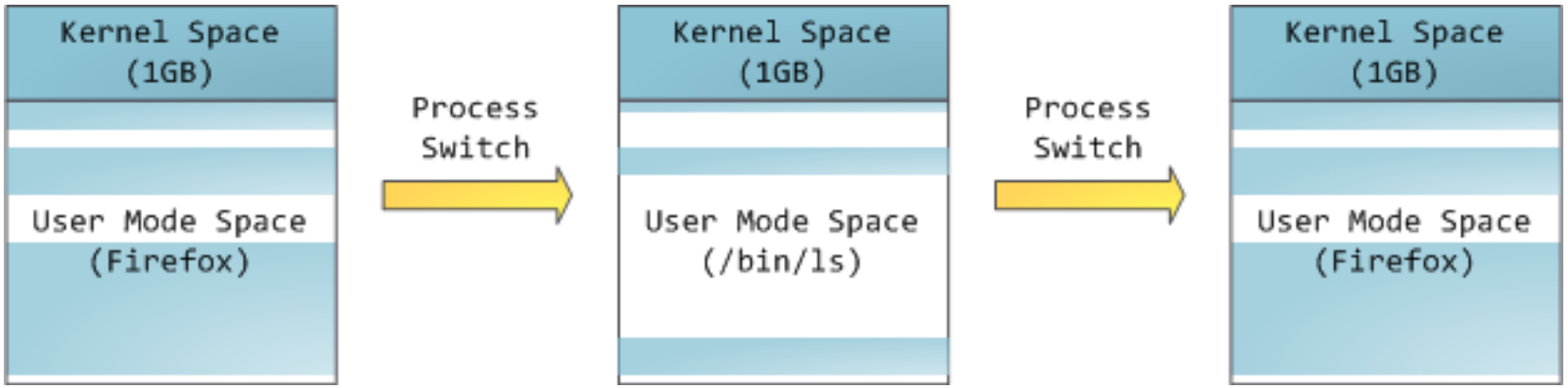
Kernel en el espacio virtual

- Cada proceso tiene su espacio de memoria virtual (con una TP asociada)
- El *kernel* también necesita un espacio virtual de direcciones
- Una porción del espacio virtual de los procesos se reserva para el *kernel*

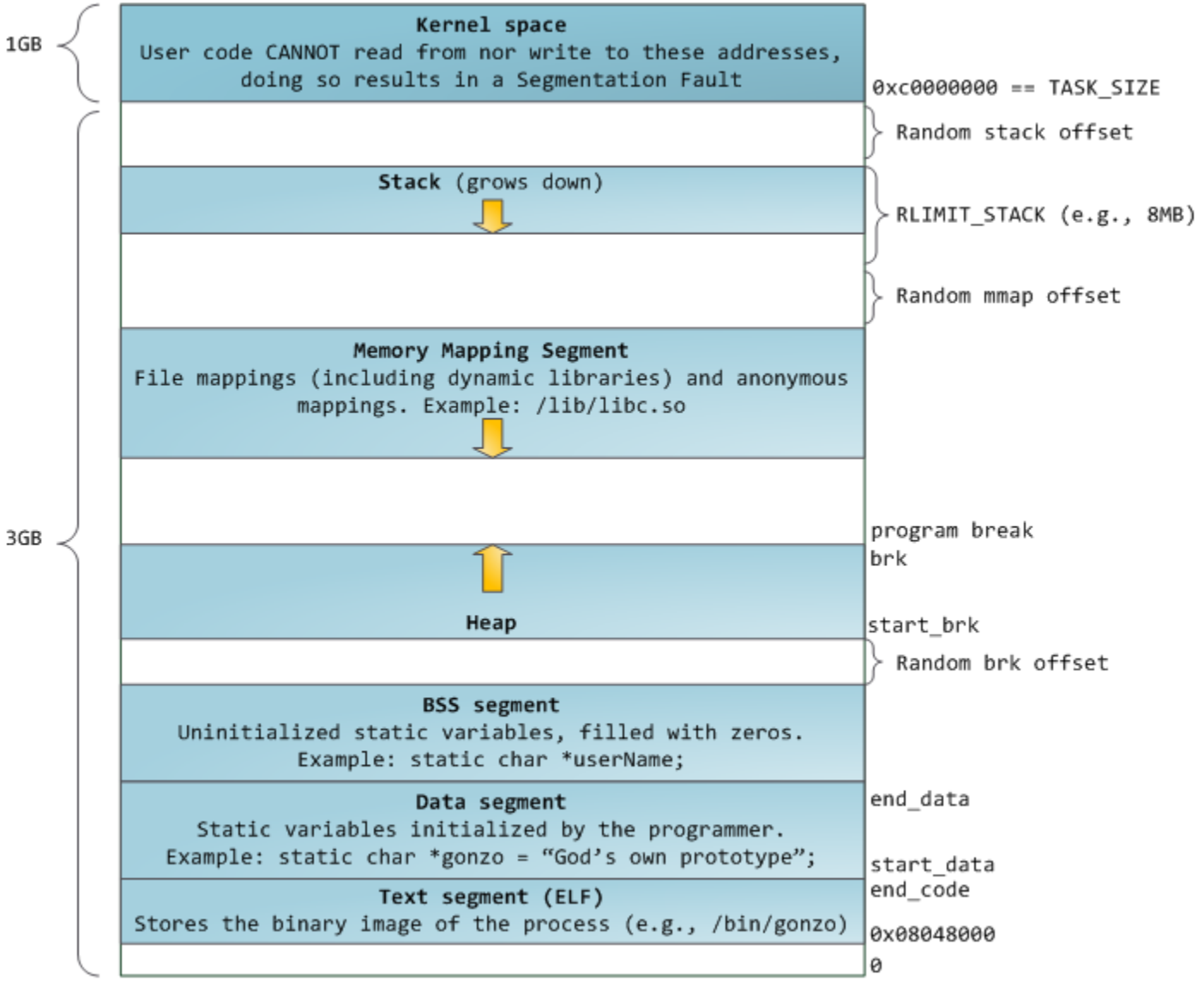


Kernel en el espacio virtual

- El *kernel* está siempre presente en la misma ventana del espacio virtual
  - Listo para responder a interrupciones y llamadas al sistema
- Al cambiar el contexto de ejecución, cambia la TP para traducir el nuevo proceso, pero la ventana del kernel permanece
- Páginas del kernel no accesibles en modo usuario



Mapa de un proceso



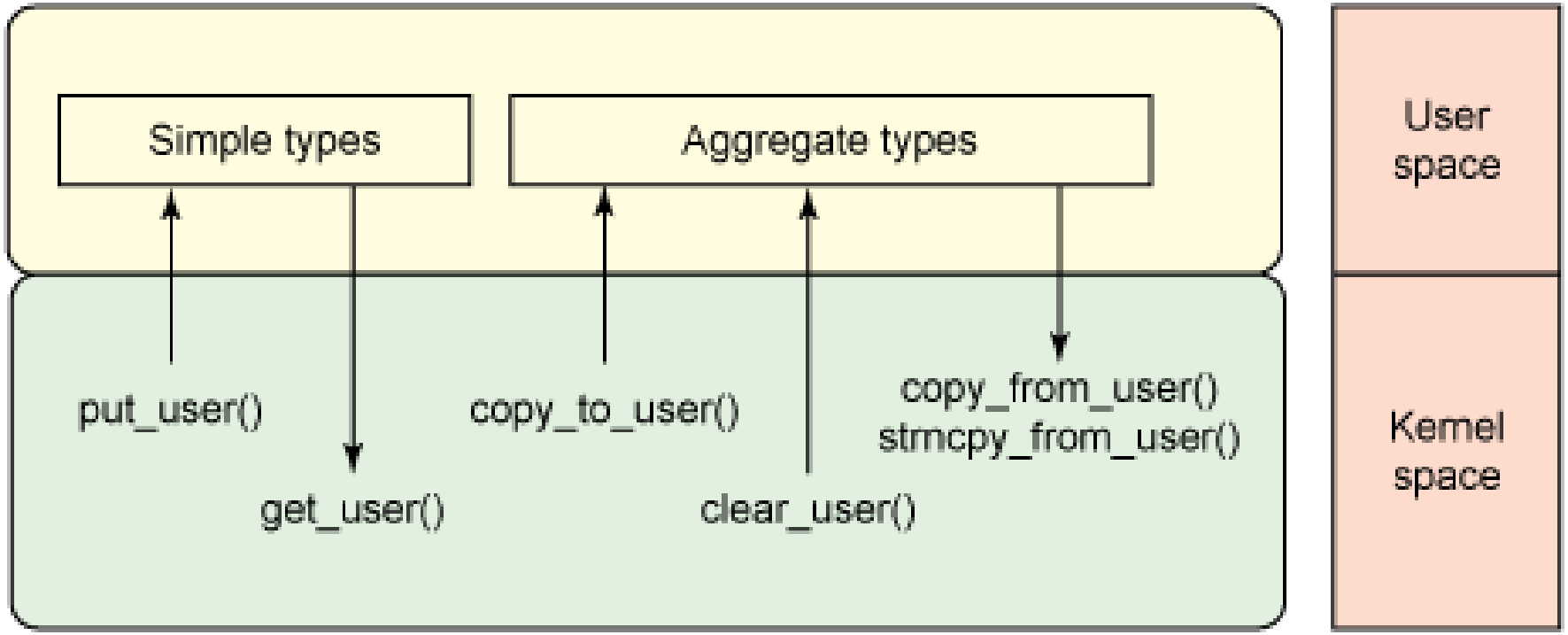
Acceso al espacio de usuario desde el kernel

- En una llamada al sistema
- Un puntero de usuario no debe ser desreferenciado por el kernel
  - Puede que el puntero de usuario no sea válido en modo kernel (arquitectura, kernel,…)
  - La memoria de usuario es virtual, no se permiten fallos de página en el kernel
  - El puntero lo suministra el usuario, puede ser erróneo o malintencionado
- Usar las funciones del kernel para realizar la transferencia

Funciones del kernel para acceso a espacio de usuario

Función	Descripción
access_ok	Checks the validity of the user space memory pointer
get_user	Gets a simple variable from user space
put_user	Puts a simple variable to user space
clear_user	Clears, or zeros, a block in user space
copy_to_user	Copies a block of data from the kernel to user space
copy_from_user	Copies a block of data from user space to the kernel
strnlen_user	Gets the size of a string buffer in user space
strncpy_from_user	Copies a string from user space into the kernel

Funciones del kernel para acceso a espacio de usuario



https://www.ibm.com/developerworks/library/l-kernel-memory-access/

API del kernel para acceso a espacio de usuario

```
#include <asm/uaccess.h>

get_user(x, ptr); // macro
put_user(x, ptr); // x: kernel var, ptr: user address

unsigned long copy_to_user(void __user * to,
                           const void * from,
                           unsigned long n);
unsigned long copy_from_user(void * to,
                             const void __user * from,
                             unsigned long n);
```

- Estas funciones/macros se pueden bloquear (fallo del página)
- Usar siempre en el contexto de un proceso de usuario (dentro de *system call*)

https://www.ibm.com/developerworks/library/l-kernel-memory-access/